# Adversarial Searching

Dr. Nirmala Shinde Baloorkar

Assistant Professor

Department of Computer Engineering, KJSSE

SOMAIYA VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Somaiya TRUST

# Outline

- Game
- MiniMax
- Alpha Beta Pruning

# Games in AI

- In AI, “games” usually refers
  - Deterministic:
    - If the next state of environment is **completely determined** by current state and action executed by the agent (not probabilistic)
  - Turn-taking:
    - 2 agents whose actions must alternate
  - Zero-sum games:
    - if one agent wins, the other loses
  - Perfect information:
    - fully observable

# Other Games

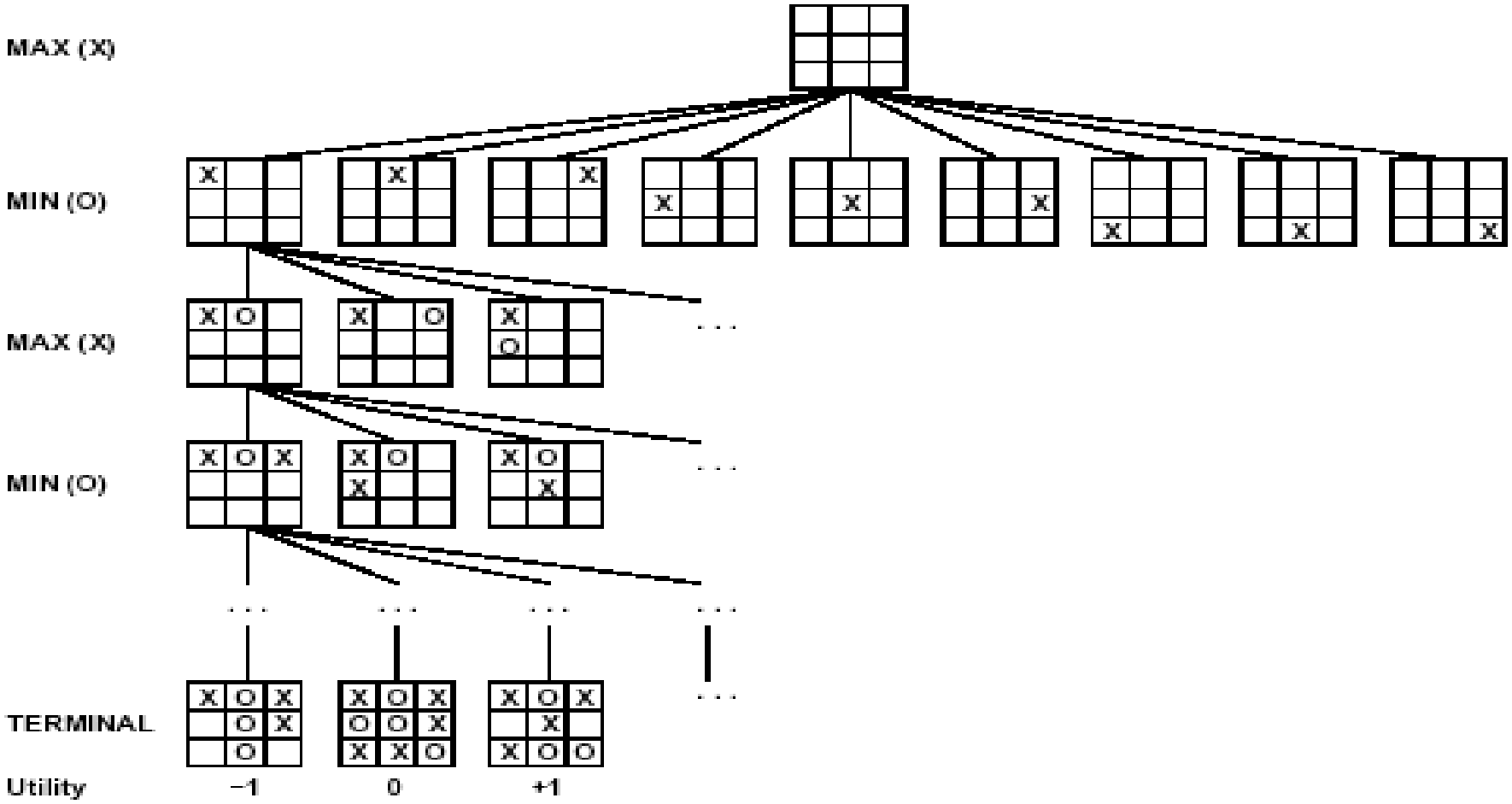
| | deterministic | chance |
|---|---|---|
| perfect information | chess, checkers, go, othello | backgammon, monopoly |
| imperfect information | stratego | bridge, poker, scrabble, nuclear war |

# Games as Search

- States:
  - board configurations
- Initial state:
  - the board position and which player will move
- Successor function:
  - returns list of (move, state) pairs, each indicating a legal move and the resulting state
- Terminal test:
  - determines when the game is over
- Utility function:
  - gives a numeric value in terminal states

  (e.g., -1, 0, +1 for loss, tie, win)

# Games as Search (continued)



# Adversarial Search

- Min Max algorithm
- Alpha beta pruning

# Adversial - Mini-Max Terminology

- move: a move by both players
- ply: a half-move
- utility function: the function applied to leaf nodes
- backed-up value
  - of a max-position: the value of its largest successor
  - of a min-position: the value of its smallest successor
- minimax procedure: search down several levels; at the bottom level apply the utility function, back-up values all the way up to the root node, and that node selects the move

# Adversarial - Mini-Max Terminology

- Adversarial Search for the minimax procedure works as follows:
  - It aims to find the optimal strategy for MAX to win the game.
  - <u>It follows the approach of Depth-first search.</u>
  - In the game tree, optimal leaf node could appear at any depth of the tree.
  - Propagate the minimax values up to the tree until the terminal node discovered.

$$\textsc{Minimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{max} \\ \min_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{min} \end{cases}$$

# Example 1

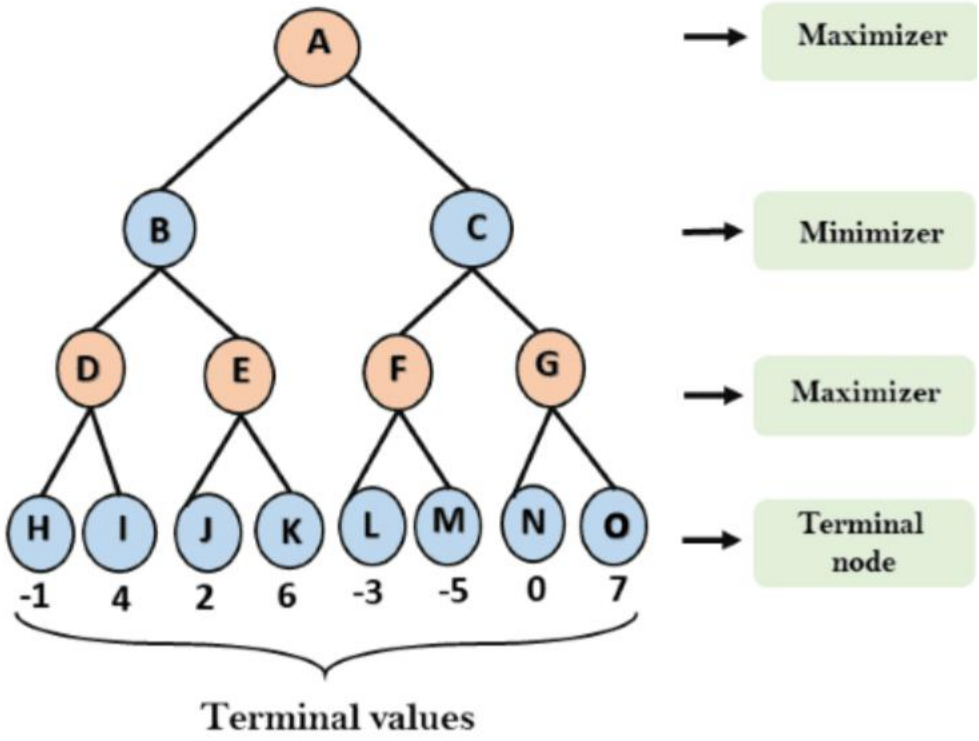
- Applies the utility function to obtain utility values for the terminal states. Let's assume the tree's initial state in the diagram below.

**Min = + ∞**

worst-case initial value = + infinity.

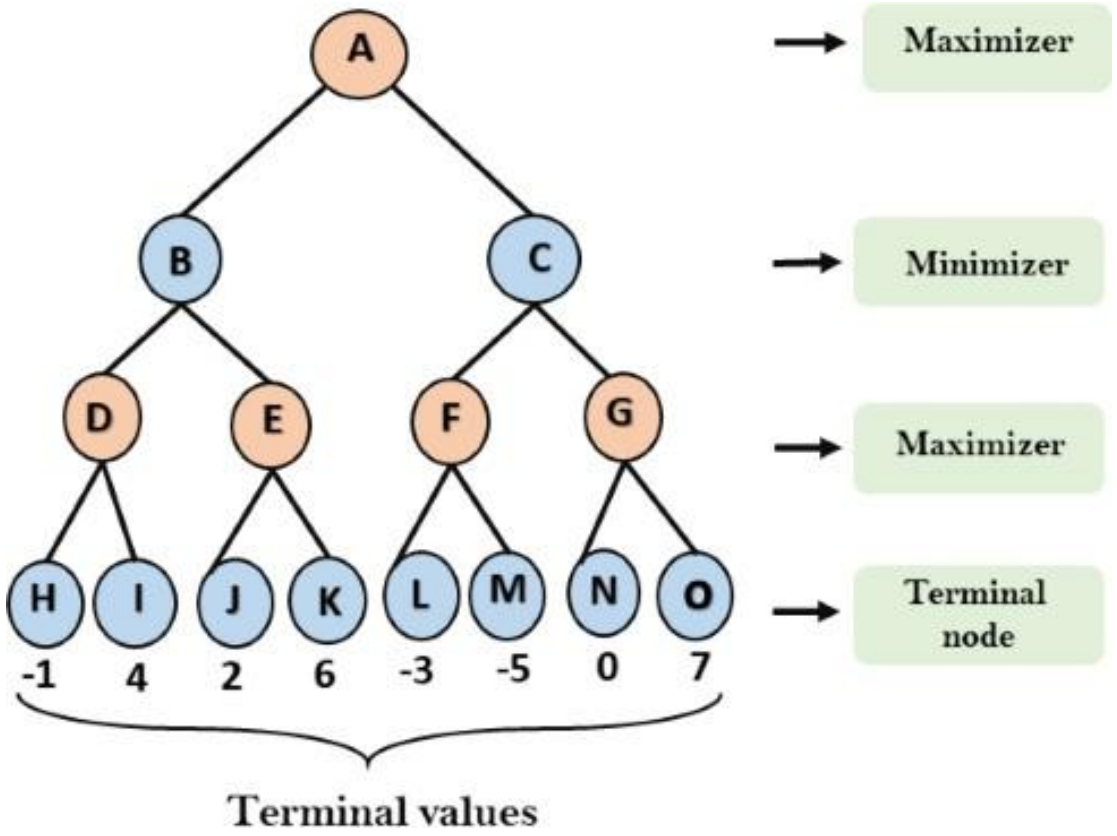
**Max = - ∞**

worst-case initial value = - infinity



# Example 1 – Step 2

The Maximizer assesses the utility value for each terminal state and compares these values against its initial utility. This allows it to determine the optimal values for the upper nodes, ultimately selecting the best possible option from all the available choices.

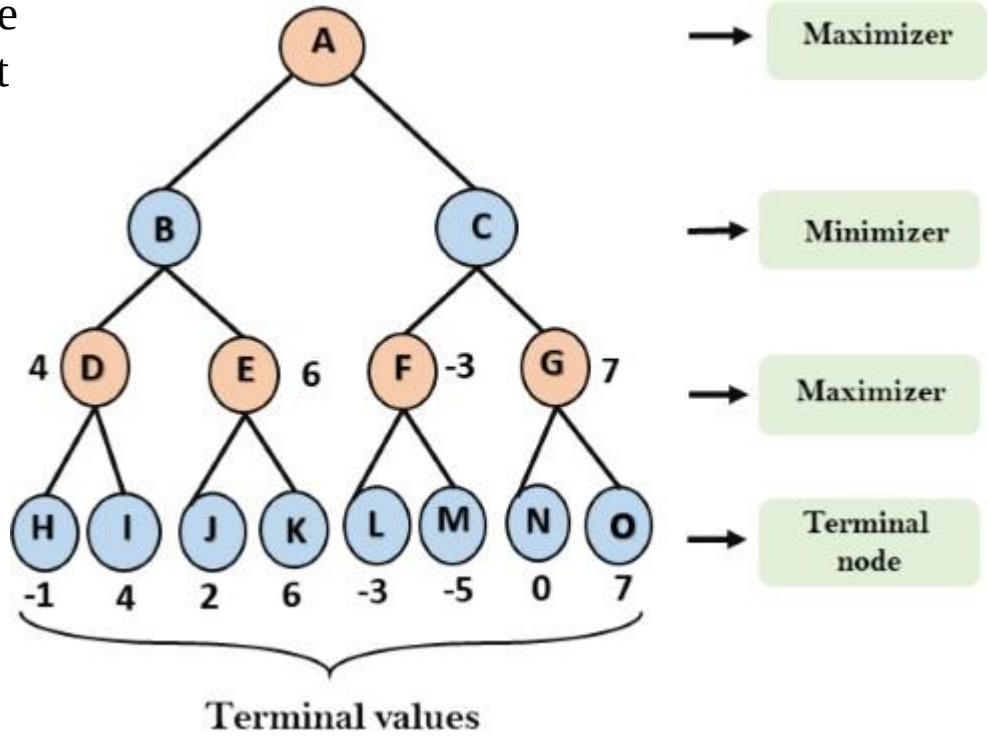



- For node D max(-1,- -∞) => max(-1,4) = 4
- For Node E max(2, -∞) => max(2, 6) = 6
- For Node F max(-3, -∞) => max(-3,-5) = -3
- For node G max(0, -∞) = max(0, 7) = 7

# Example 1 – Step 3

Now, the Minimizer assesses the utility values of the current nodes and uses them to calculate the values for the nodes in the layer above. This process helps determine the best outcomes at the next higher level of the decision tree

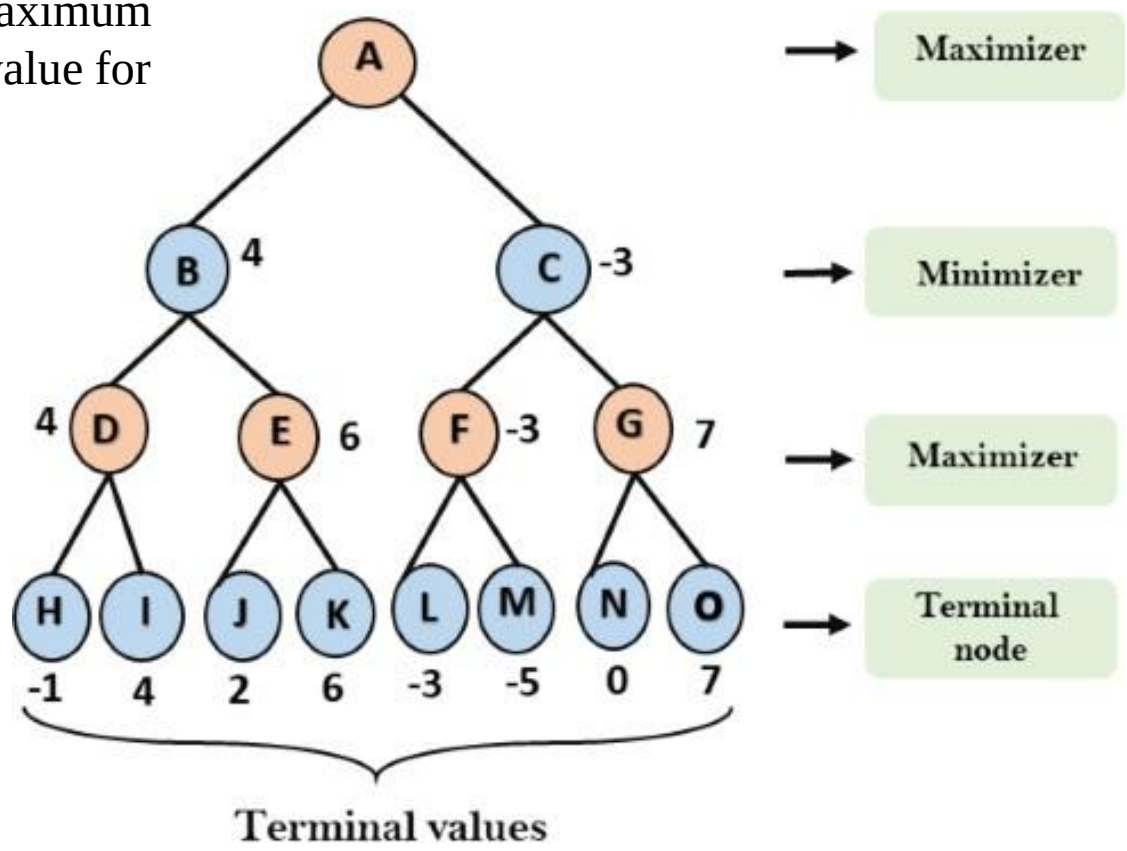
For node B= min(4,6) = 4
For node C= min (-3, 7) = -3



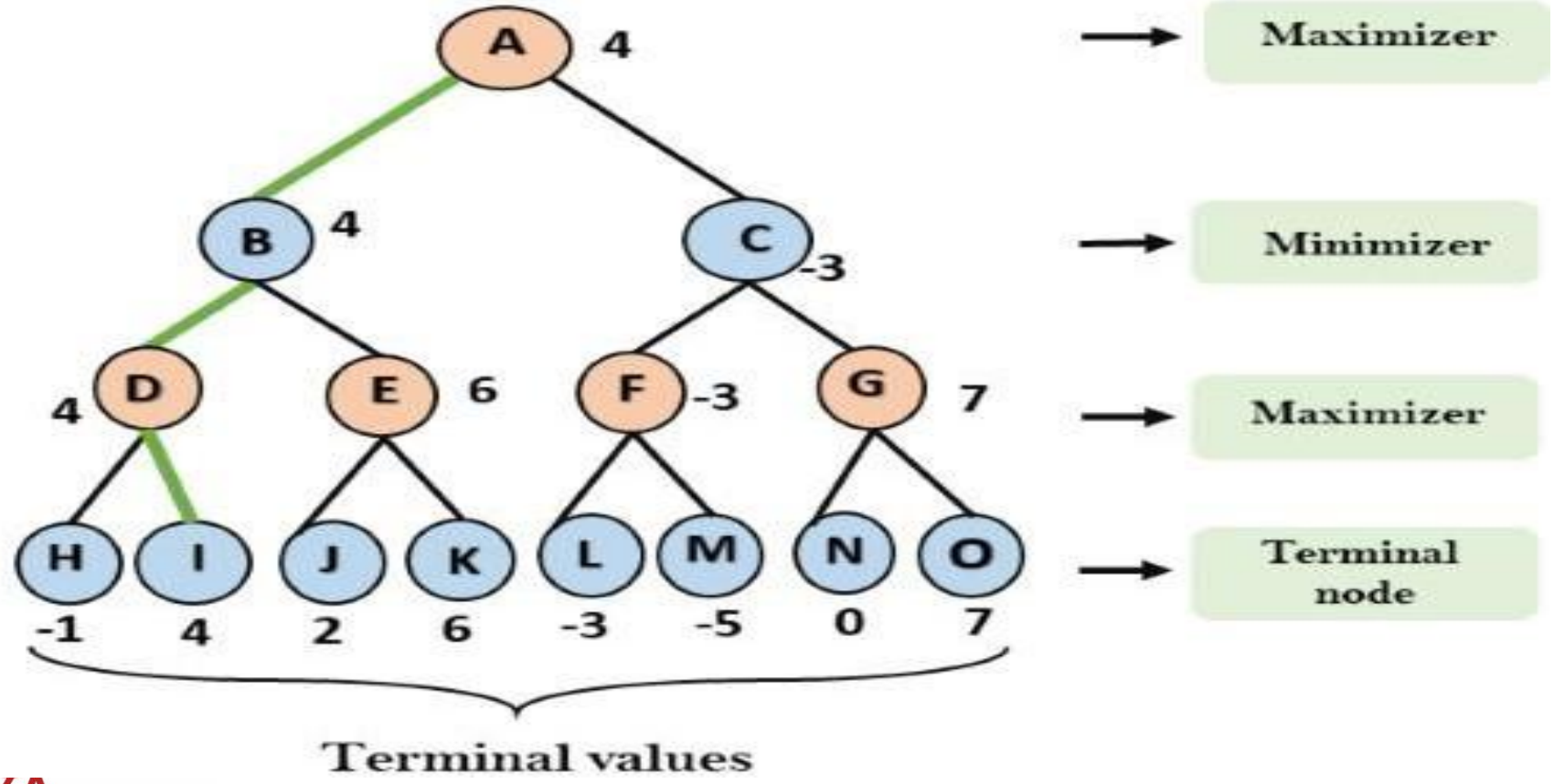
# Example 1 – Step 4

Now, it's the Maximizer's turn to act. It evaluates the maximum value among all the nodes to determine the maximum value for the root node

For node A= max(4,-3) = 4



# Example 1 - Solution



# MiniMax Algorithm

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))
```

---

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v
```

---

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

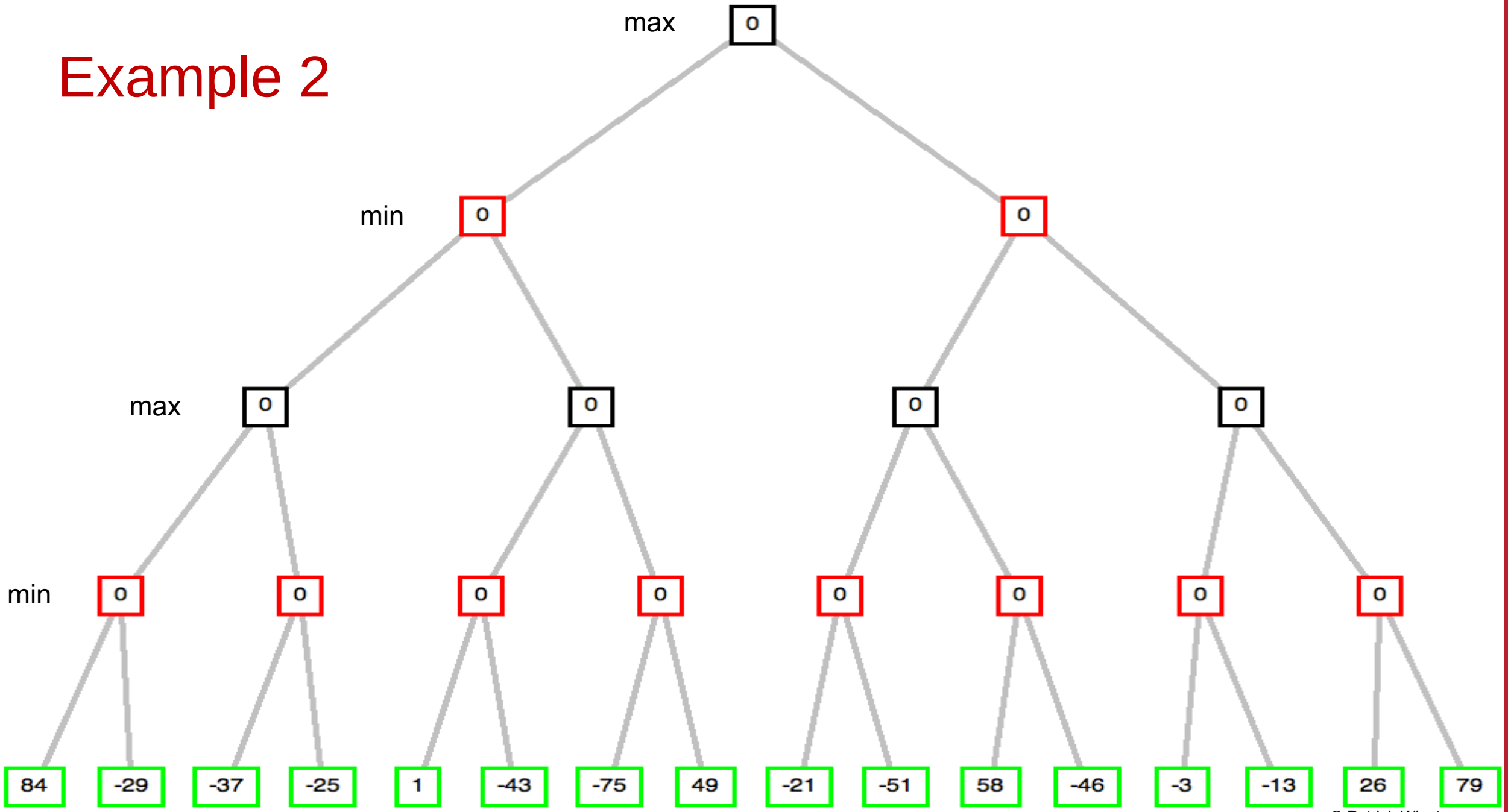
# Mini-Max Properties

- Complete? Yes, if tree is finite
- Optimal?
  - Against an optimal opponent? Yes
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$

# Application of MiniMax

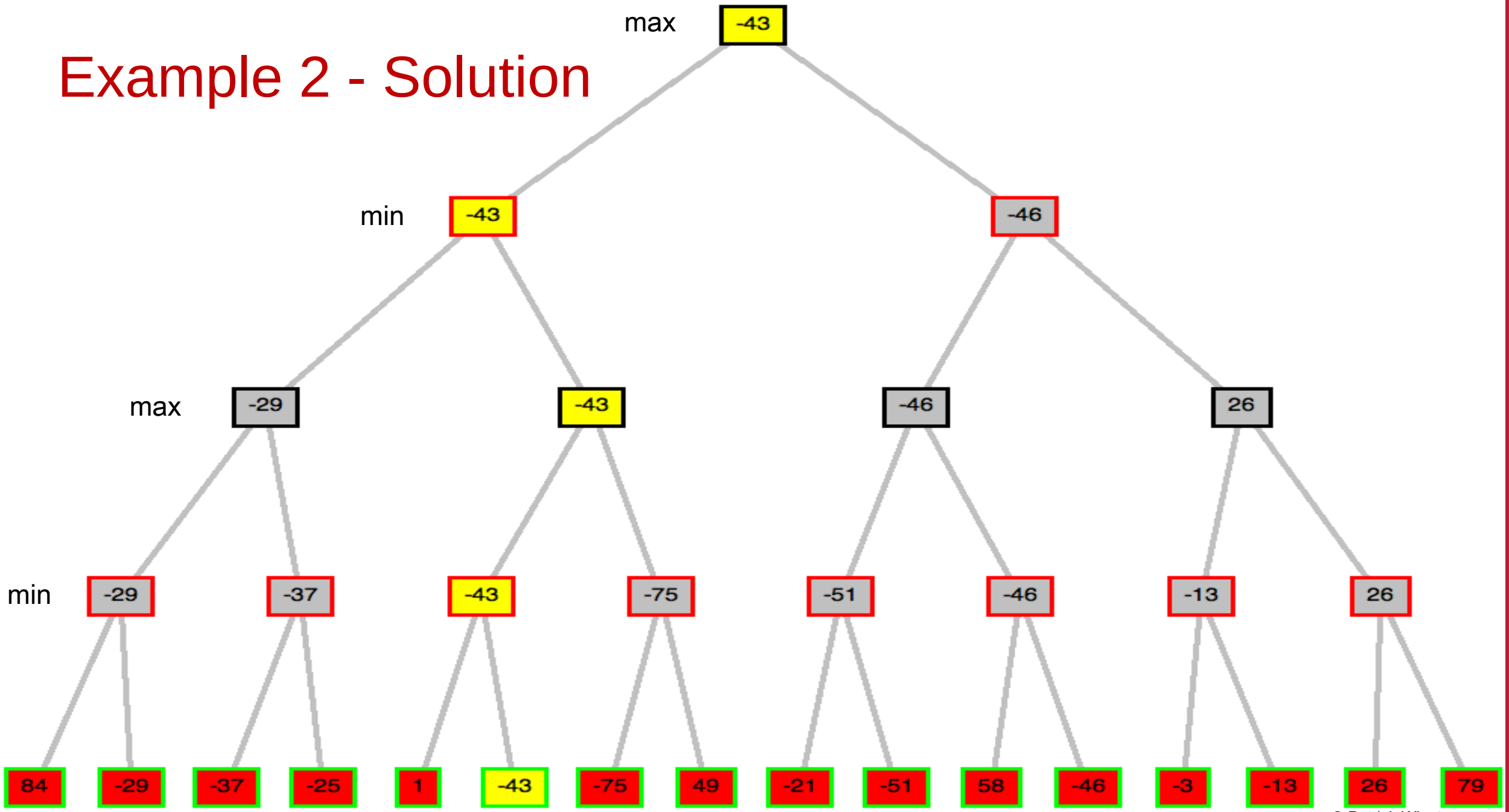
- **Game Theory**
- **AI in Robotics**
- **Economics**
- **Decision-making in AI**

# Example 2




© Patrick Winston

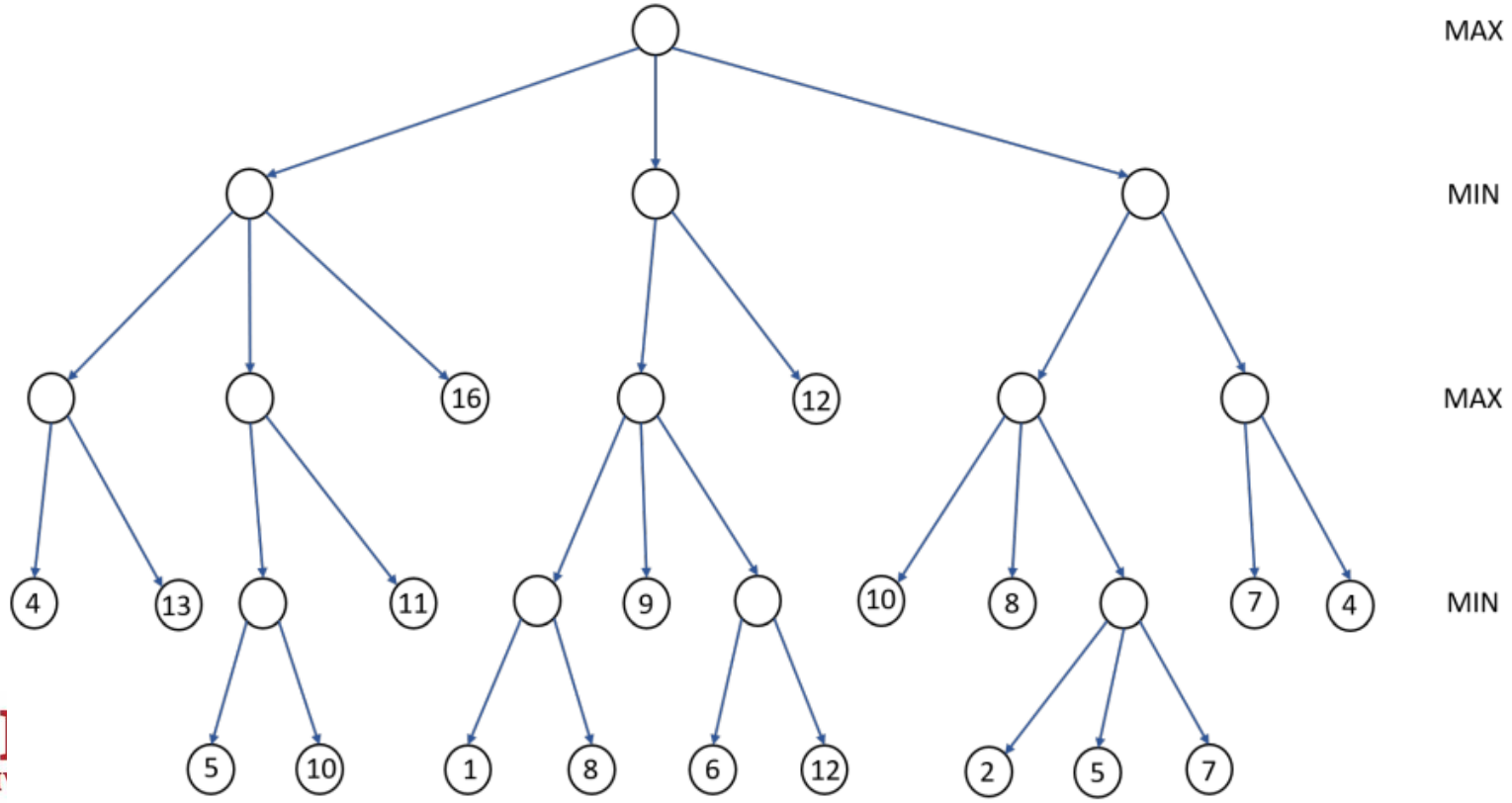
# Example 2 - Solution



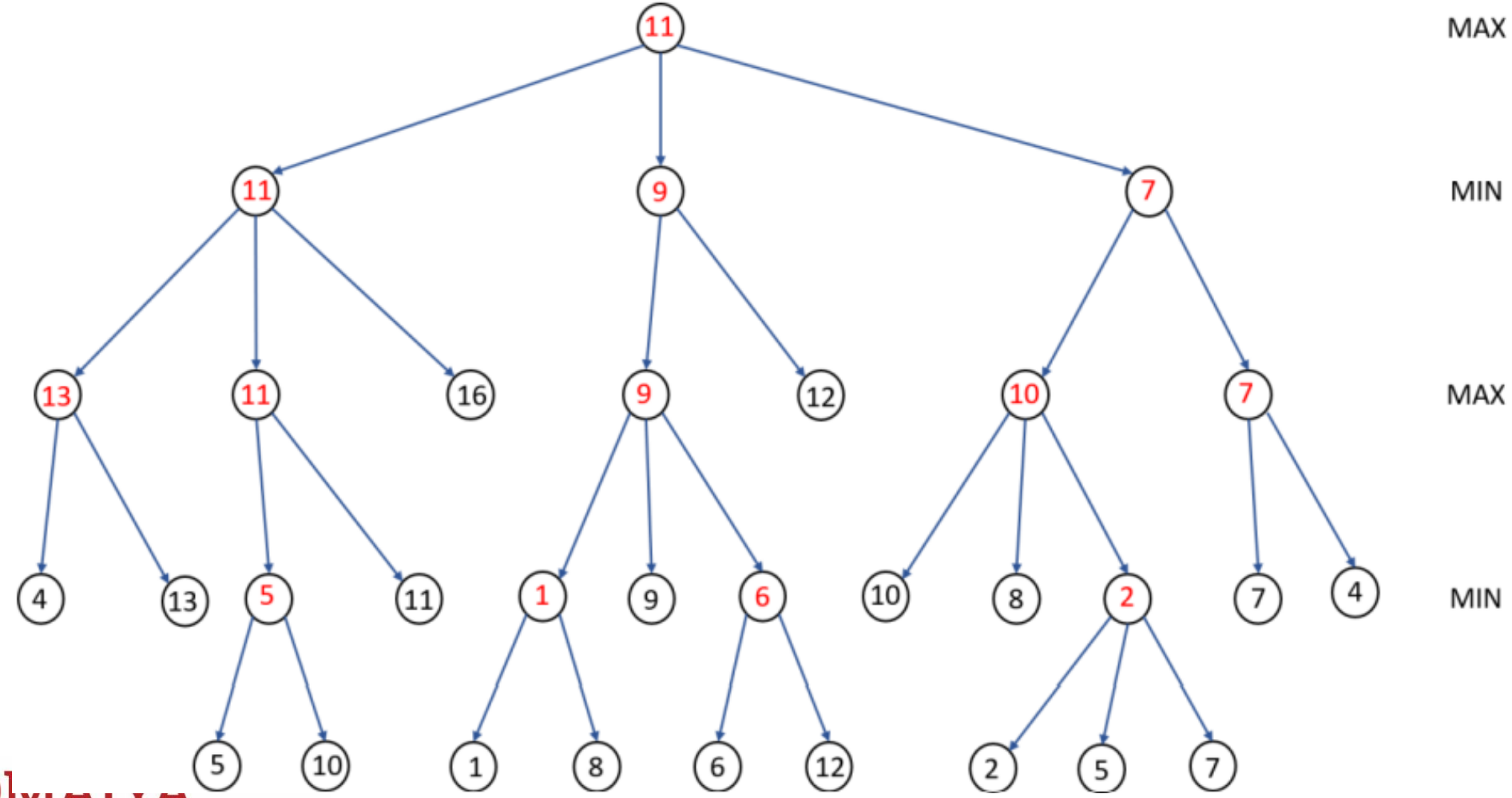

© Patrick Winston

# Example 3

- Use the Minimax algorithm to compute the minimax value at each node for the game tree below

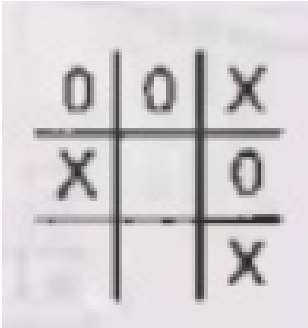


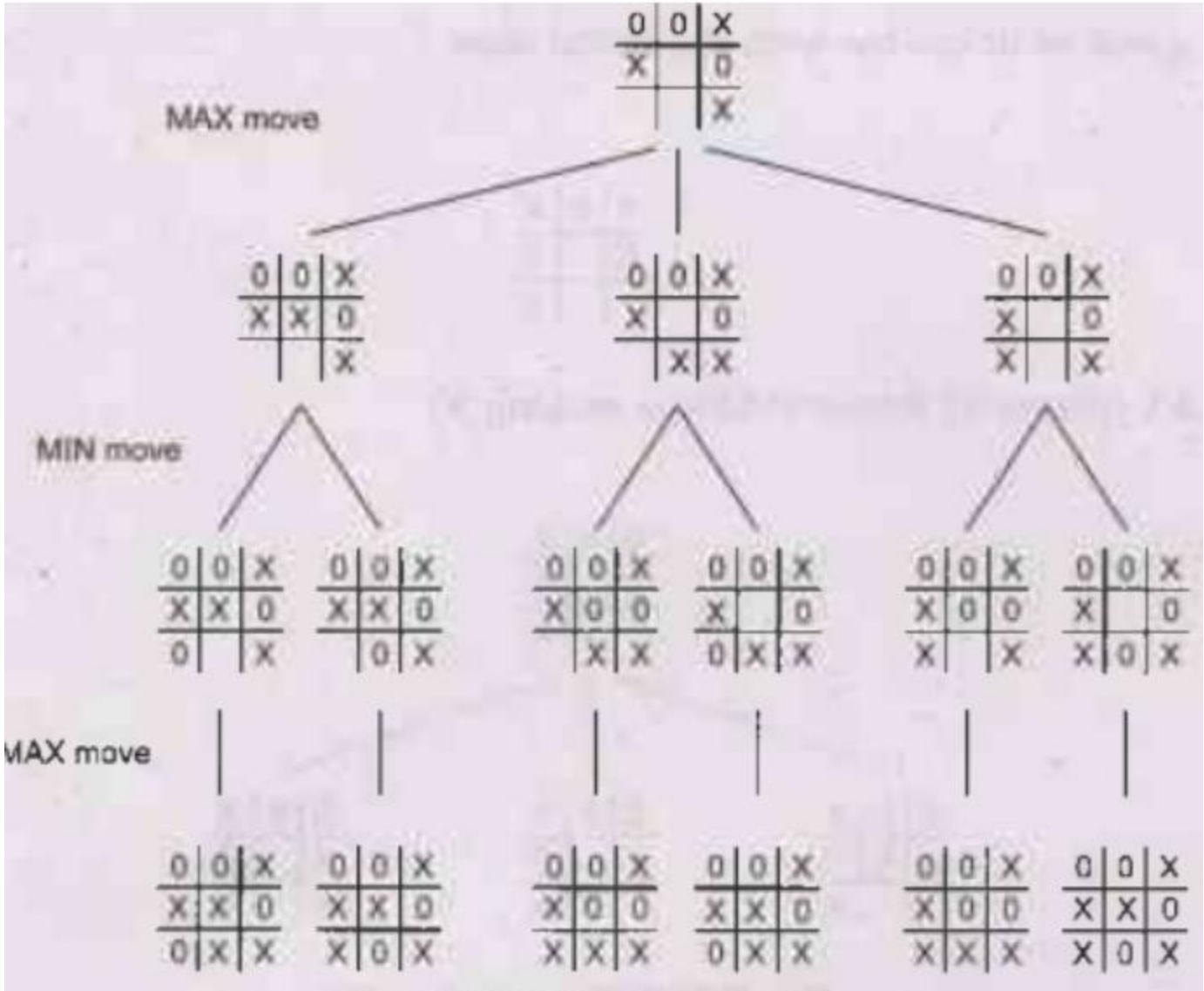
# Example 3 - Solution



# Min Max Example

- Initial State




MAX move
MIN move
MAX move

# Alpha-Beta Pruning

- Optimization technique for Minimax algorithm
- This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**
- The two-parameter can be defined as:
  - **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is -∞.
  - **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is +∞.

# Condition for Alpha-beta pruning: α>=β

The **Max player** will only update the value of **alpha**.

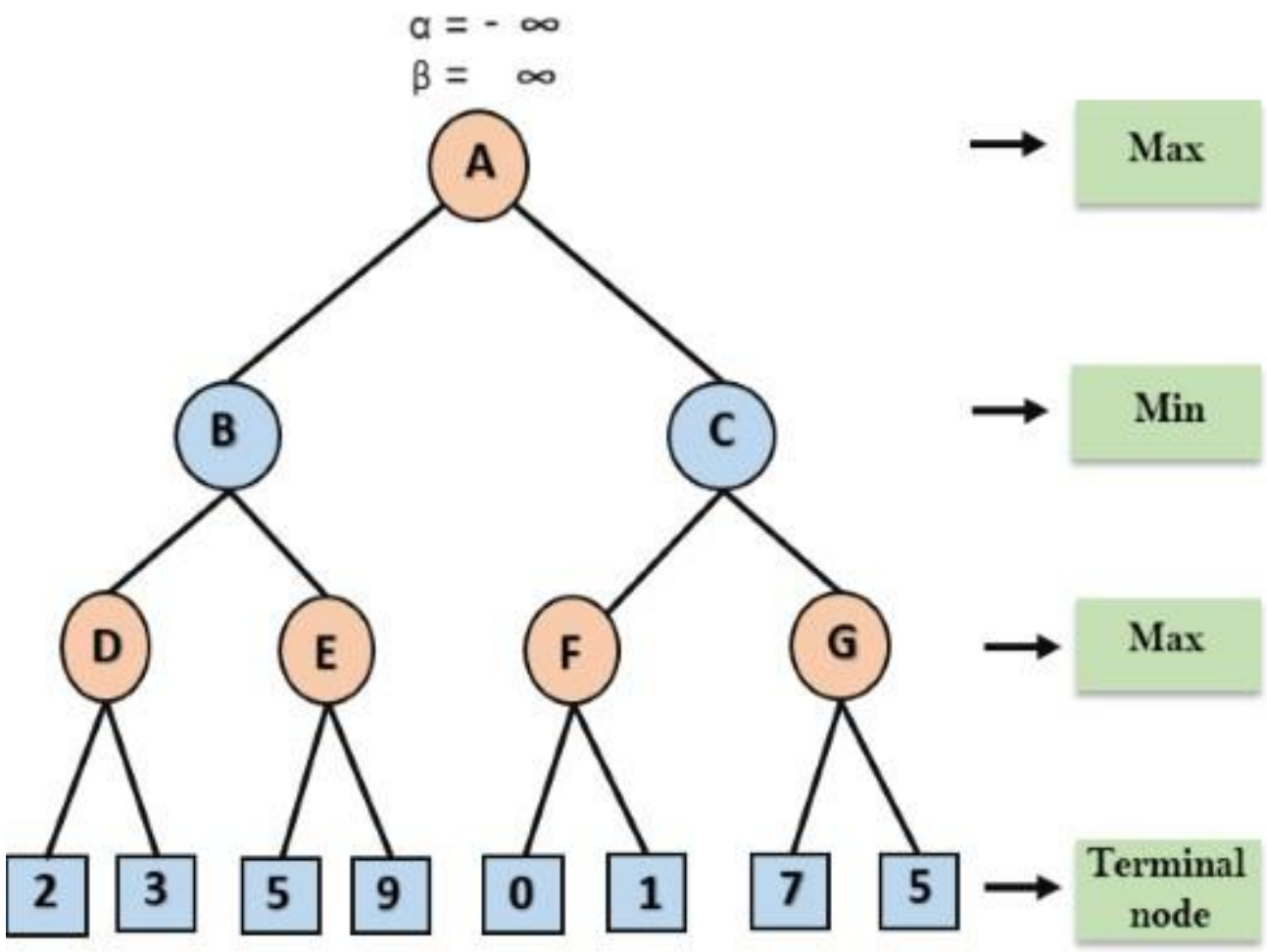
The **Min player** will only update the value of **beta.**

- We will only pass the alpha, beta values to the child nodes.
- No values will be sent upwards the tree Depth First Approach with Backtracking.

# Alpha–Beta Pruning Algorithm (Steps)

1. Start at root node (MAX level)
2. Initialize $\alpha = -\infty$ and $\beta = +\infty$
3. Traverse tree depth-first (left → right)
4. At MAX node: update $\alpha = \max(\alpha, \text{child value})$
5. At MIN node: update $\beta = \min(\beta, \text{child value})$
6. If $\alpha \geq \beta$ → prune remaining subtree
7. Continue until all necessary nodes evaluated
8. Return optimal value at root (same as Minimax)

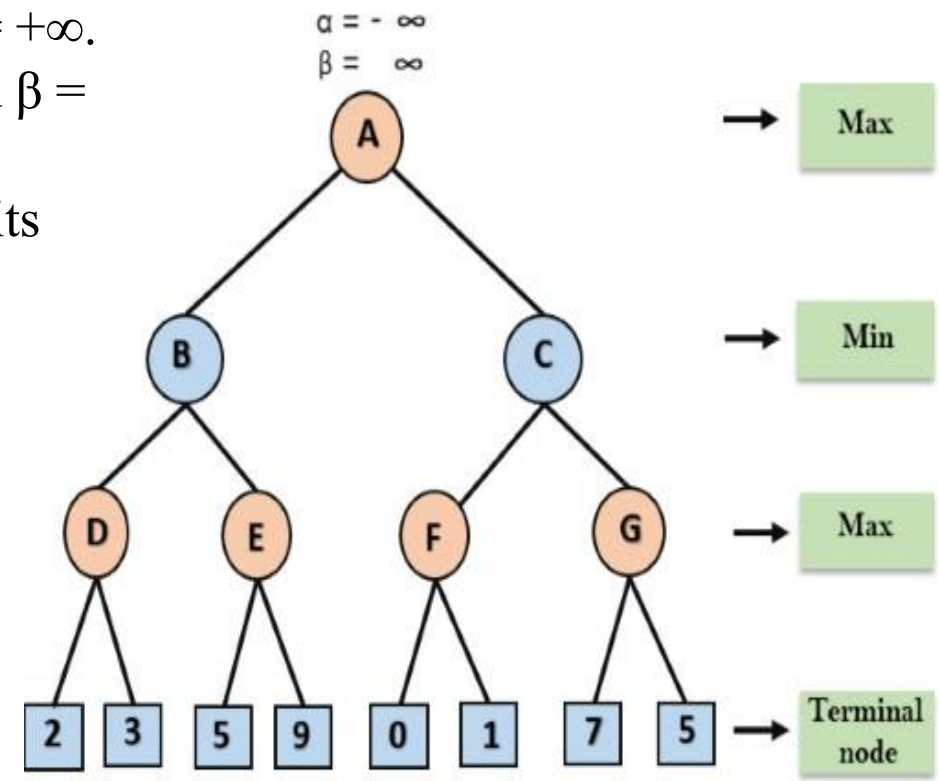
# Example 4



# Example 4

The Max player starts at node A, with α = -∞ and β = +∞. These values are passed to node B, where α = -∞ and β = +∞ remain unchanged.
Node B then forwards the same values of α and β to its child node D.

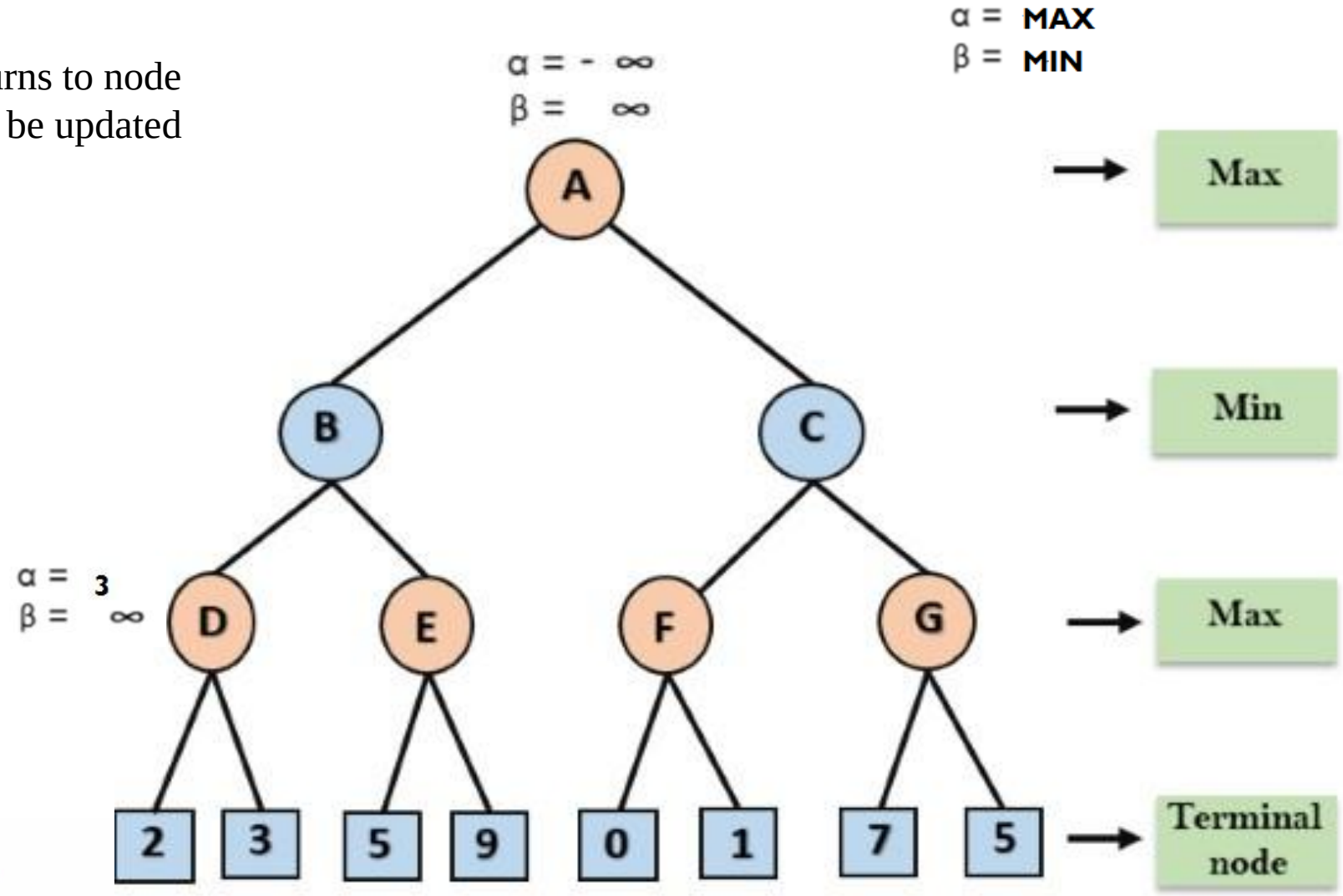
The value at Node D will be determined during Max's turn.



# Example 4 – Step 1

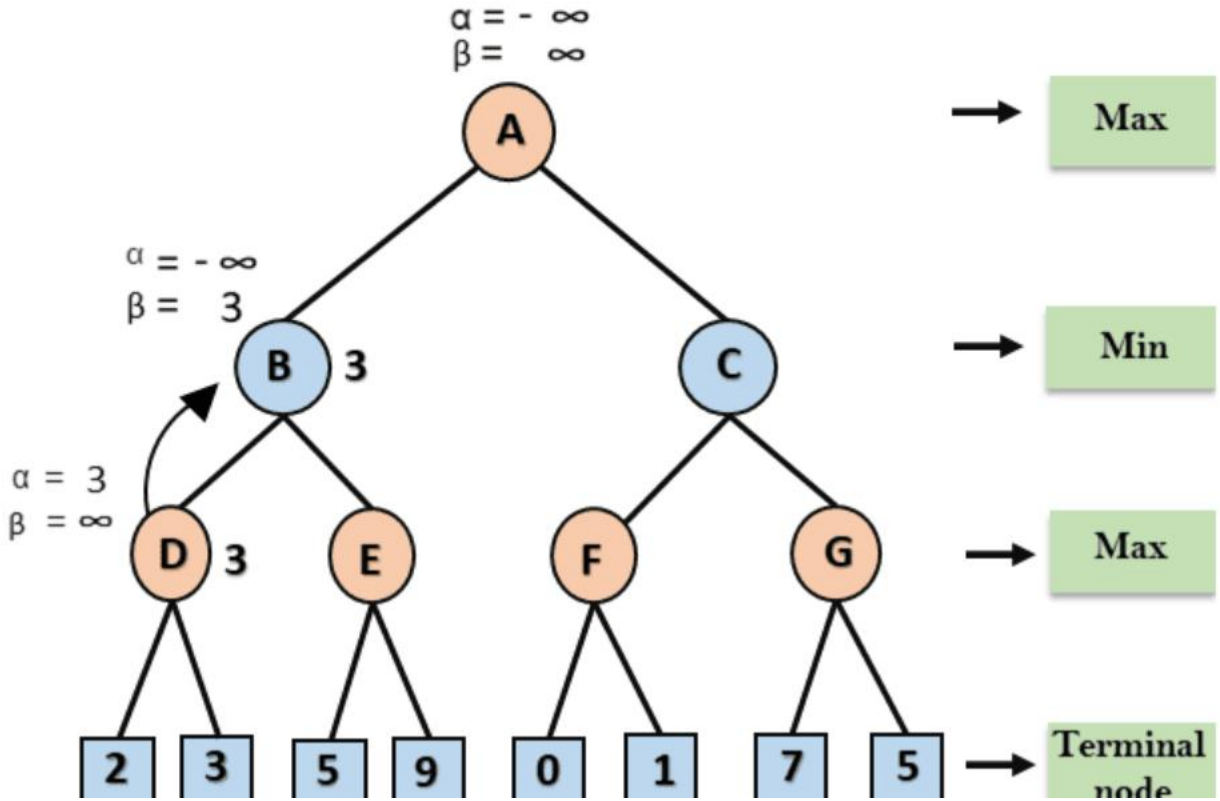
The algorithm now returns to node B, where the value will be updated since it is Min's turn.

At Node D.
Leaf nodes: 2, 3
α = max(−∞, 2) = 2
α = max(2, 3) = 3
Return value(D) = 3

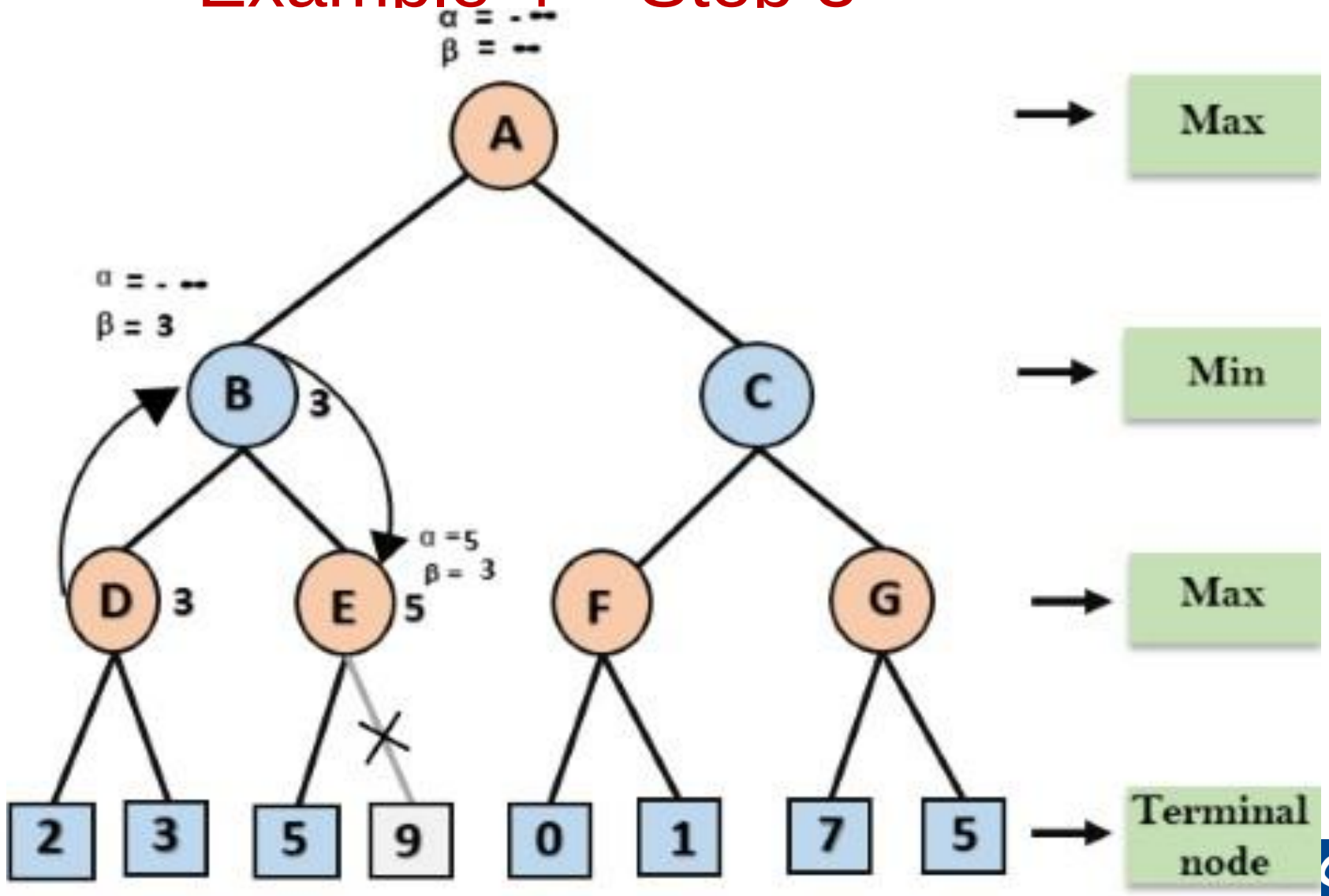


# Example 4 – Step 2

Evaluate Node D (MAX)
Children: 2, 3
Evaluate left to right: First leaf = 2
$\alpha=\max(-\infty,2)=2$ α=max(−∞,2)=2
Second leaf = 3
$\alpha=\max(2,3)=3$ α=max(2,3)=3
Return value of D = 3
Back to B (MIN)
$\beta=\min(+\infty,3)=3$
β=min(+∞,3)=3



# Example 4 – Step 3

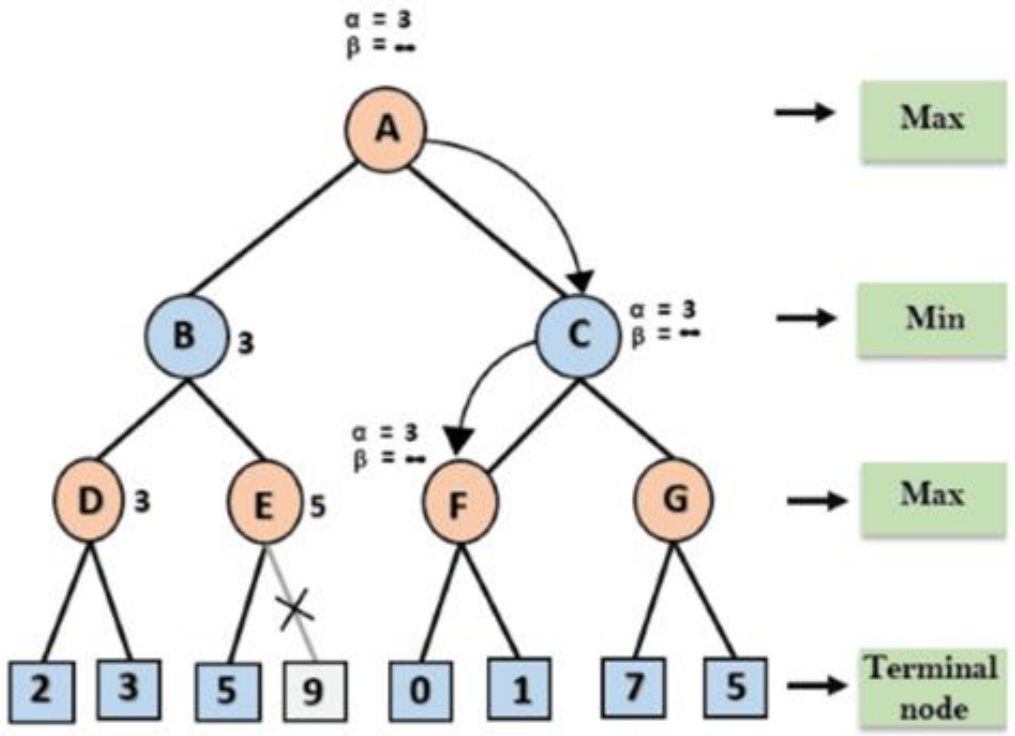


# Example 4 – Step 4

- The method now backtracks in the tree, moving from node B to node A.
- At node A, the value of α is updated to the highest available value, resulting in **$\alpha = \max(-\infty, 3) = 3$**, while β remains unchanged at $+\infty$. These updated values, $\alpha = 3$ and $\beta = +\infty$, are then passed to A's right successor, node C.
- At node C, $\alpha = 3$ and $\beta = +\infty$ are forwarded to its child node, F, without any changes.

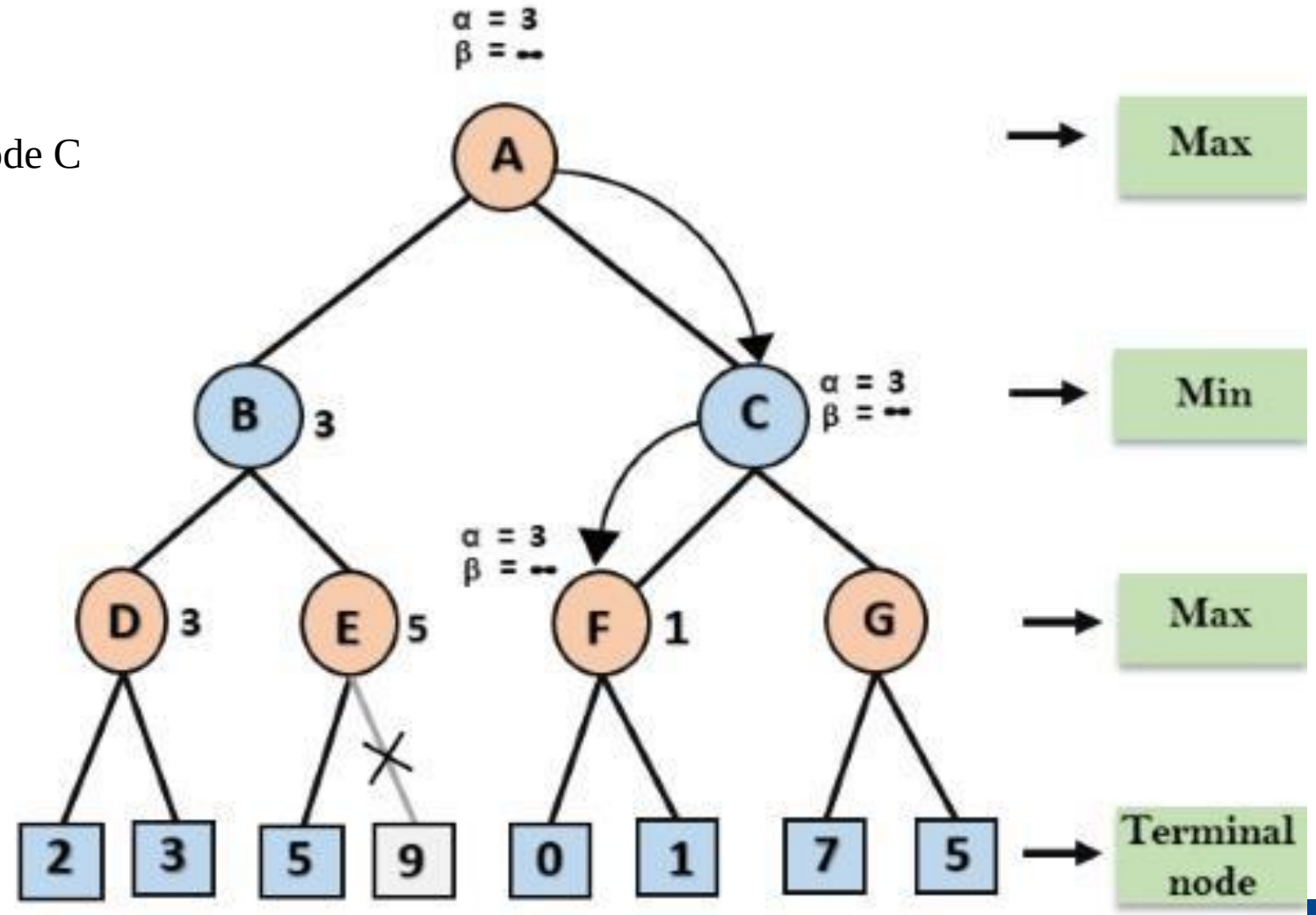
# Example 4 – Step 5

At node F, the value of α is compared with its left child, which has a value of 0.



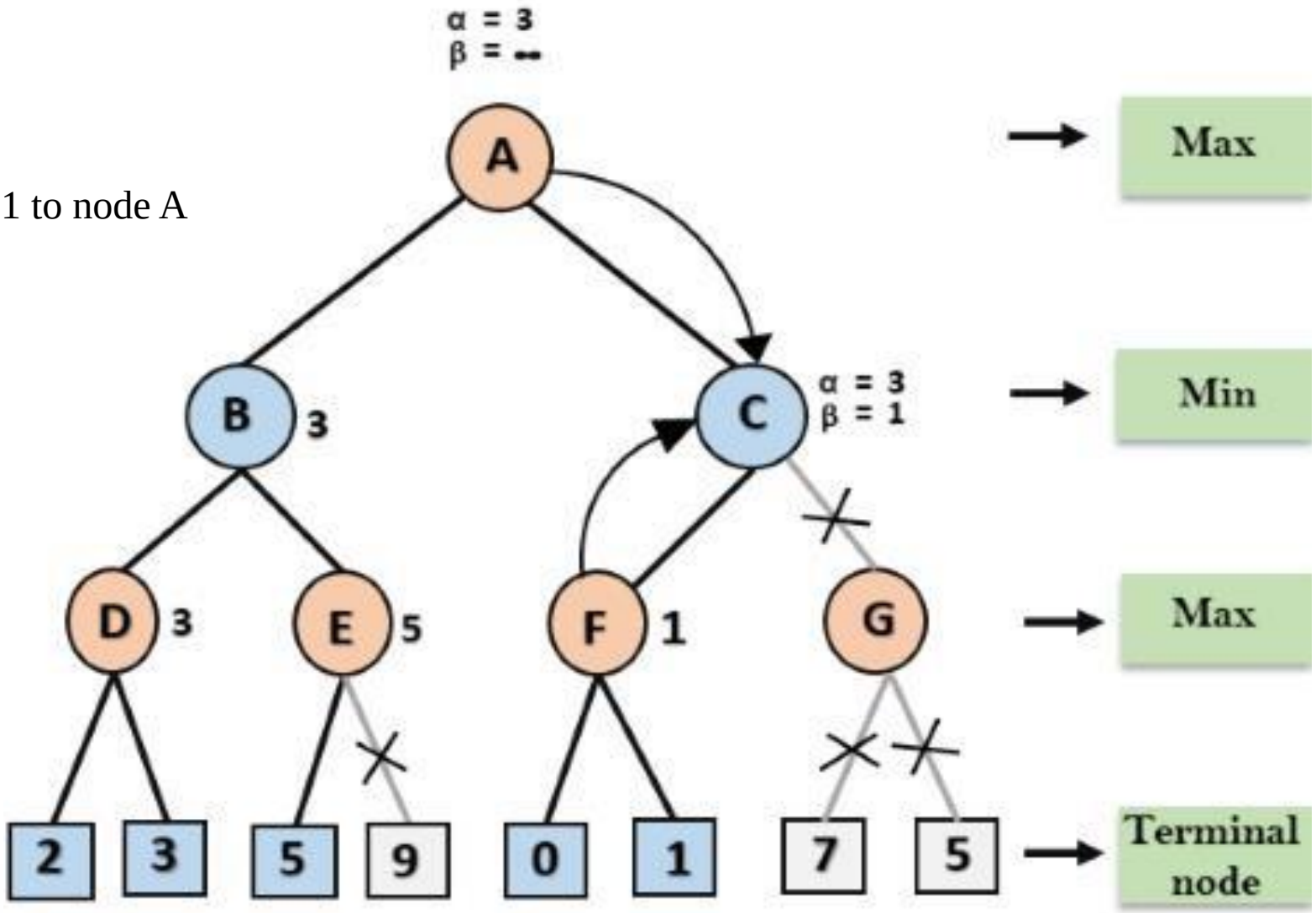
# Example 4 – Step 6

Node F returns the node value of 1 to node C

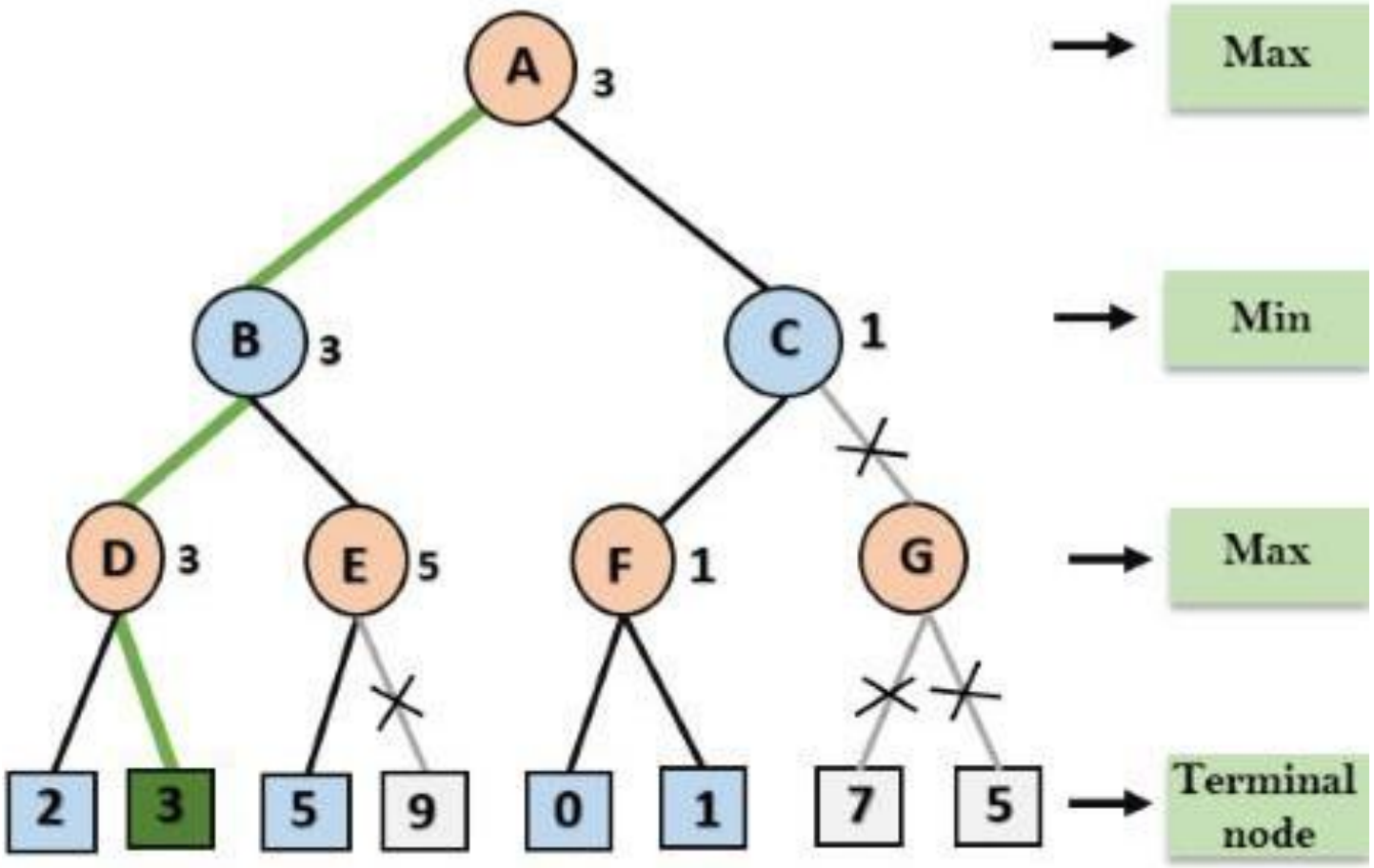


# Example 4 – Step 7

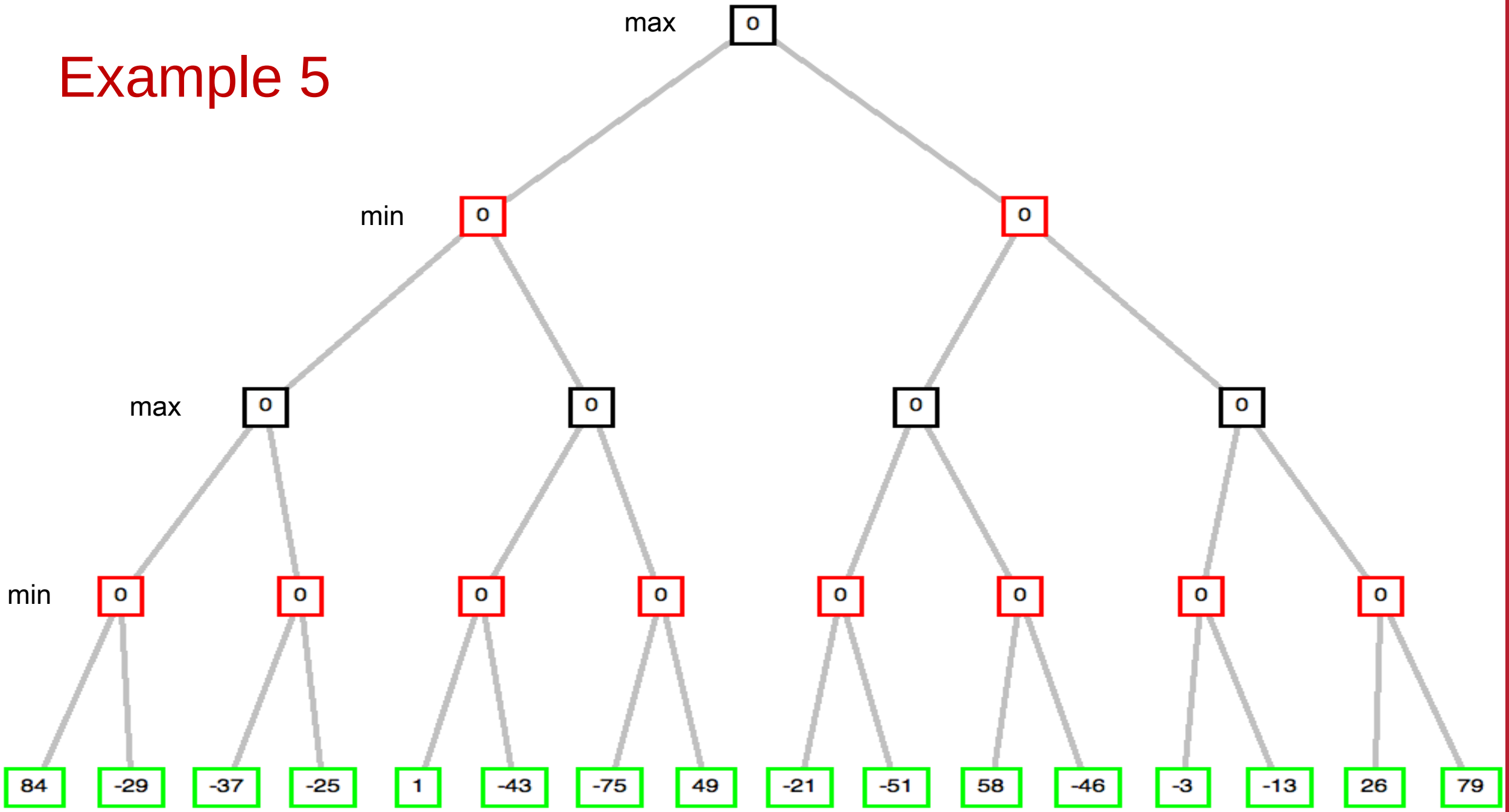
Node C returns the value of 1 to node A



# Example 4 - Solution



# Example 5


max
0
min
0
0
max
0
0
0
0
min
0
0
0
0
0
0
0
0
84
-29
-37
-25
1
-43
-75
49
-21
-51
58
-46
-3
-13
26
79

# Alpha-Beta

```
MinVal(state, alpha, beta){
  if (terminal(state))
      return utility(state);
  for (s in children(state)){
   child = MaxVal(s,alpha,beta);
   beta = min(beta,child);
   if (alpha>=beta) return child;
  }
  return beta; }
```

**alpha** = the **highest** value for **MAX** along the path

**beta** = the **lowest** value for **MIN** along the path

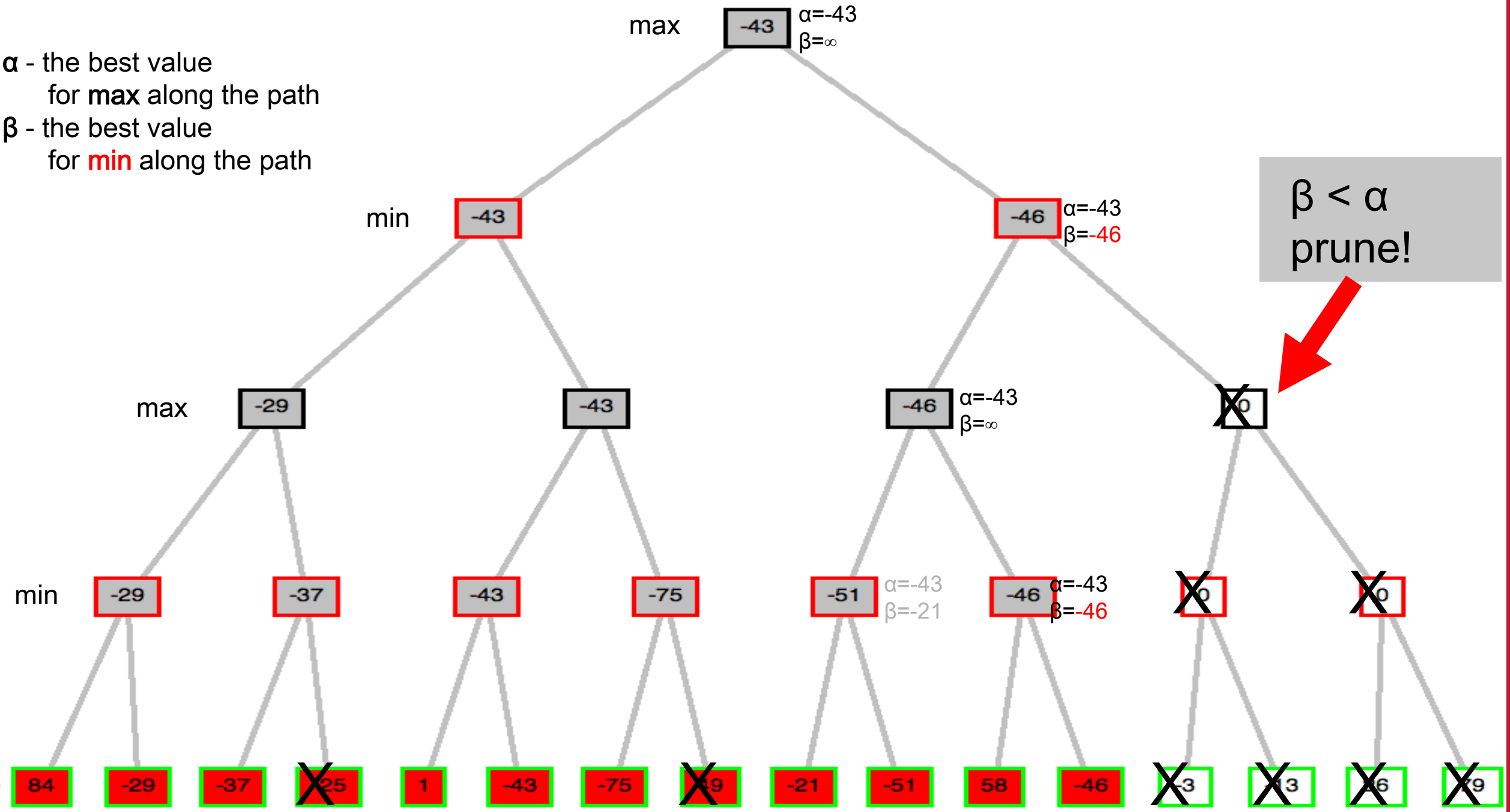
# Alpha-Beta

```
MaxVal(state, alpha, beta){
  if (terminal(state))
      return utility(state);
  for (s in children(state)){
   child = MinVal(s,alpha,beta);
   alpha = max(beta,child);
   if (alpha>=beta) return child;
  }
  return beta; }
```

**alpha** = the **highest** value for **MAX** along the path

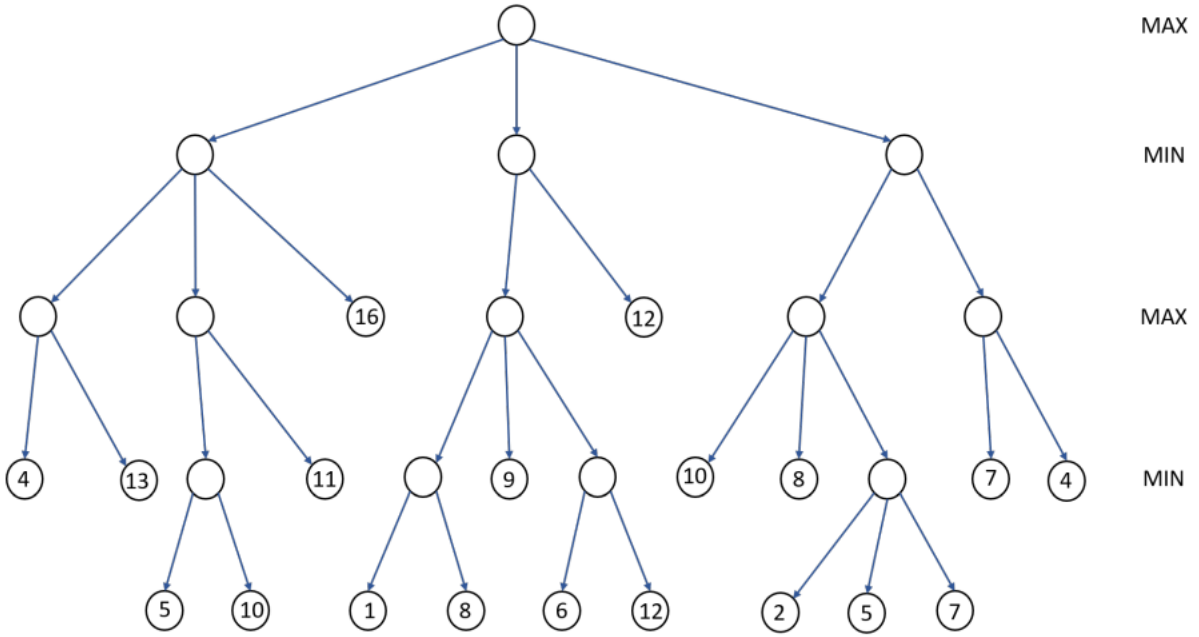
**beta** = the **lowest** value for **MIN** along the path

α - the best value
for **max** along the path
β - the best value
for min along the path



# Example 6

- Use the alpha beta pruning algorithm for the game tree below



# Examples

- https://pywix.blogspot.com/2020/05/alpha-beta-pruning-in-artificial.html
- https://tutorialforbeginner.com/alpha-beta-pruning-in-ai
- https://www.youtube.com/watch?v=egNkn_-uelA
- https://www.youtube.com/watch?v=DCNP5L9_t4E
- https://pages.cs.wisc.edu/~dyer/cs540/hw-toc.html - IMP

- Best Video
- https://www.cse.iitd.ac.in/~mausam/courses/col333/autumn2025/